Lecture Notes: Transformer Architectures

Course: 2110573 Pattern Recognition

2026-03-17
version 1.0

1 Introduction to the Transformer

The Transformer is a novel network architecture introduced in the 2017 paper "Attention Is All You Need" by Vaswani et al. from Google [27]. It was a significant departure from the recurrent neural network (RNN) based models that were state-of-the-art for sequence transduction tasks like machine translation at the time.

The core idea of the Transformer is to rely exclusively on **attention mechanisms**, completely discarding recurrence and convolutions. This design has two primary advantages:

- **Increased Parallelization:** By removing the sequential nature of RNNs, computations over a sequence can be performed in parallel, leading to significantly faster training times.
- **Improved Long-Range Dependency Handling:** The attention mechanism allows the model to directly connect any two positions in a sequence, regardless of their distance. This makes it more effective at capturing long-range dependencies compared to RNNs, where information has to pass through many intermediate steps.

2 High-Level Architecture of Vanilla Transformer

The Transformer follows a standard **encoder-decoder** structure, which is common in sequence-to-sequence tasks.

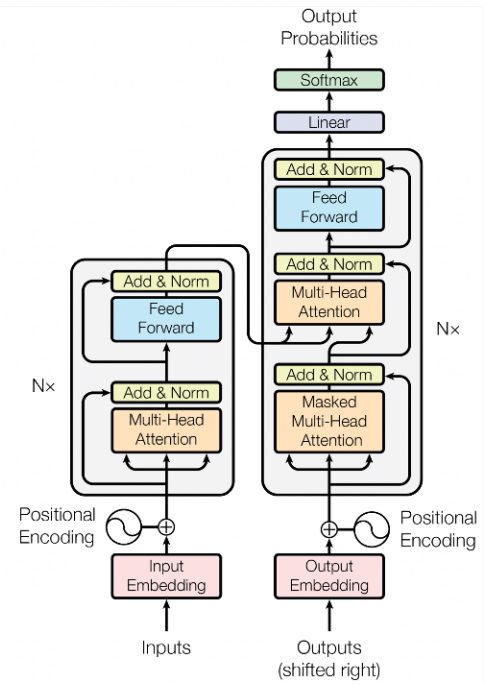

Figure 1: The Transformer Architecture, showing the stack of N=6 encoders and decoders. Source: "Attention is All You Need".

2.1 The Encoder-Decoder Stack

The architecture consists of a stack of identical encoders and a stack of identical decoders. In the original paper, the number of layers (N) in each stack is 6.

- **The Encoder:** Its role is to map an input sequence of symbol representations $(x_1, \ldots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \ldots, z_n)$.
- **The Decoder:** Given $\mathbf{z}$, the decoder generates an output sequence $(y_1, \ldots, y_m)$ one element at a time. It is auto-regressive, meaning it consumes the previously generated symbols as additional input when generating the next one.

2.2 Layer Composition

Each encoder and decoder layer is composed of a few key sub-layers.

- **Encoder Layer:** Each encoder layer has two sub-layers:
  1. A multi-head self-attention mechanism.
  2. A simple, position-wise fully connected feed-forward network.
- **Decoder Layer:** Each decoder layer has three sub-layers:
  1. A **masked** multi-head self-attention mechanism to prevent positions from attending to subsequent positions.
  2. A multi-head **encoder-decoder attention** mechanism, which allows every position in the decoder to attend over all positions in the input sequence.
  3. A position-wise fully connected feed-forward network.

A residual connection followed by layer normalization is applied around each of these sub-layers.

3 The Attention Mechanism

Attention can be described as a function that maps a query and a set of key-value pairs to an output. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.1 Scaled Dot-Product Attention

The specific attention mechanism used in the Transformer is called Scaled Dot-Product Attention. For a query Q, keys K, and values V, the attention is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

Where d_k is the dimension of the keys. The scaling factor $\sqrt{d_k}$ is crucial for stabilizing gradients, as the dot products can grow large for high-dimensional vectors, pushing the softmax function into regions with extremely small gradients.

3.2 Attention Variations in the Transformer

The Transformer uses three different types of attention:

1. **Self-Attention (Encoder):** Here, the queries, keys, and values (Q, K, V) all come from the same place: the output of the previous layer in the encoder. This allows each position in the encoder to attend to all other positions in the previous layer's output, effectively calculating a representation of the sequence by relating its own words to each other.

2. **Masked Self-Attention (Decoder):** In the decoder's self-attention layer, queries, keys, and values also come from the same source (the previous decoder layer). However, to preserve the auto-regressive property (i.e., prediction of the current token can only depend on previous tokens), future positions are masked out. This is implemented by setting the softmax input for illegal connections to $-\infty$.

3. **Encoder-Decoder Attention (Decoder):** This is also known as "cross-attention". Here, the queries Q come from the previous decoder layer, while the keys K and values V come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence, which is how the decoder focuses on relevant parts of the source sentence.

4 Multi-Head Attention

Instead of performing a single attention function, the authors found it beneficial to linearly project the queries, keys, and values multiple times with different, learned linear projections. This is called Multi-Head Attention. It allows the model to jointly attend to information from different representation subspaces at different positions.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2}$$

where each head is calculated as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{3}$$

The process is as follows:

1. The input Q, K, and V matrices are multiplied by learned weight matrices (W_i^Q, W_i^K, W_i^V) for each of the h attention heads.

2. Scaled dot-product attention is applied in parallel to each of these projected versions of Q, K, and V, yielding h output matrices $(Z_0, Z_1, \ldots, Z_{h-1})$.

3. These output matrices are concatenated.

4. The concatenated matrix is multiplied by another learned weight matrix W^O to produce the final output of the multi-head attention layer.

In the base Transformer model, $h = 8$.

5 Positional Encodings

Since the model contains no recurrence or convolution, it has no inherent sense of the order or position of the tokens in the input sequence. To address this, "positional encodings" are added to the input embeddings at the bottoms of the encoder and decoder stacks. These encodings are vectors that give the model information about the position of a token in the sequence.

The authors used sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \tag{4}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}}) \tag{5}$$

where pos is the position of the token in the sequence, i is the dimension within the embedding vector, and d_{model} is the model's embedding dimension. This choice allows the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

There are other types of positional encoding, such as learned absolute encodings (used in GPT [19]) and relative positional encodings (used in GPT-NeoX [3]).

6 Add & Norm: Residuals and Layer Normalization

Each sub-layer (self-attention, feed-forward network) in the encoder and decoder has a residual connection around it, followed by a layer normalization step. The output of each sub-layer is thus $\text{LayerNorm}(x + \text{Sublayer}(x))$.

Layer Normalization differs from Batch Normalization. Layer Normalization computes the mean and variance used for normalization from all of the summed inputs to the neurons in a single layer for a single training example. In contrast, Batch Normalization computes these statistics across the mini-batch for each feature independently. In the context of NLP, this means Layer Normalization normalizes across the features of each word vector independently for each sample in the batch.

7 Final Output Generation

The decoder stack outputs a vector of floats. To turn this vector into an output token, a final linear layer followed by a softmax function is used.

- **Linear Layer:** A fully connected neural network that projects the vector produced by the decoder stack into a much larger vector, known as a logits vector. The size of this vector is equal to the size of the vocabulary.
- **Softmax Layer:** The softmax function turns the logit scores into probabilities. The cell with the highest probability is chosen, and the word associated with it is produced as the output token for that time step.

8 The Transformer Family: Architectural Variants

Since its inception, the Transformer architecture has been adapted into three main variants, each suited for different types of tasks.

8.1 Encoder-Only Models (Auto-Encoders)

These models use only the encoder stack of the Transformer. Because they lack a decoder, they are not naturally suited for generation. However, their self-attention mechanism is fully bidirectional, meaning every token can attend to every other token in the sequence (both past and future).

- **Examples:** BERT [8], ALBERT [14], RoBERTa [17].
- **Use Cases:** Excellent for understanding content. Used in tasks like text classification, named entity recognition (NER), and question answering.
- **Pre-training Objective:** Typically trained on a **Masked Language Model (MLM)** objective, where some percentage of input tokens are masked, and the model must predict the original tokens.

8.2 Decoder-Only Models (Autoregressive)

These models use only the decoder stack of the Transformer. The self-attention mechanism is "causal" or "masked," meaning a token can only attend to itself and previous tokens in the sequence. This makes them inherently suitable for generating sequences from left to right.

- **Examples:** GPT series [1], Llama [26], BLOOM [28].
- **Use Cases:** Primarily used for text generation, language modeling, and building chatbots.
- **Pre-training Objective:** Trained to predict the **next token** in a sequence given all previous tokens.

8.3 Encoder-Decoder Models (Sequence-to-Sequence)

These models use the full architecture of the original Transformer. They are ideal for tasks that require transforming an input sequence into a new output sequence.

- **Examples:** T5 [20], BART [15], MASS [23], the original Transformer [27].
- **Use Cases:** Machine translation, text summarization, question answering.
- **Pre-training Objective:** Varies, but often involves corrupting the input sequence in some way (e.g., masking spans of text as in T5 and BART) and training the decoder to reconstruct the original, uncorrupted sequence.

9 Challenges and Optimizations

The original Transformer architecture, while powerful, has several limitations, primarily related to its computational cost and context length. The quadratic complexity of the attention mechanism ($O(L^2)$ for a sequence of length L) is a major bottleneck.

9.1 Parameter Efficiency: ALBERT [14]

ALBERT ("A Lite BERT") tackles the large number of parameters in models like BERT. Its two main innovations are:

- **Factorized Embedding Parameterization:** It decouples the size of the vocabulary embedding (V) from the hidden layer size (H). Instead of a large embedding matrix of size $V \times H$, it uses two smaller matrices of size $V \times E$ and $E \times H$, where $E \ll H$. This reduces the parameter count from $O(V \times H)$ to $O(V \times E + E \times H)$.
- **Cross-Layer Parameter Sharing:** It shares all parameters (feed-forward networks and attention) across all layers, which dramatically reduces the total number of unique parameters and stabilizes training.

9.2 Extending Context: Rotary Position Embedding (RoPE) [24]

Standard absolute or learned positional embeddings do not generalize well to sequence lengths longer than those seen during training. **Rotary Position Embedding (RoPE)** provides a novel way to encode position that has become central to modern LLMs like Llama [26], Qwen [29], Gemma [25], and PaLM [4].

Instead of adding positional information to the embeddings, RoPE *modifies* the query and key vectors by rotating them based on their absolute position. The key insight is to perform this rotation in a way that the subsequent dot product for attention only depends on the *relative* distance between the tokens.

Mechanism: RoPE treats the d-dimensional embedding vector as a collection of $d/2$ independent two-dimensional subspaces. This decomposition allows the model to leverage the geometric properties of vectors on a 2D plane. By rotating these 2D subspaces independently and then combining their contributions utilizing the linearity of the inner product. For a query vector $\mathbf{q}_m$ at position m, the vector is split into pairs $(q_{m,2i}, q_{m,2i+1})$. Each pair is rotated by an angle determined by the position m and a frequency θ_i:

$$\begin{pmatrix} q'_{m,2i} \\ q'_{m,2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} q_{m,2i} \\ q_{m,2i+1} \end{pmatrix} \tag{6}$$

The frequencies are pre-set constants following a geometric progression:

$$\theta_i = 10000^{-2i/d}, \quad i \in [0, 1, \ldots, d/2 - 1] \tag{7}$$

This specific setting of θ_i ensures that the rotation speeds vary across dimensions, which provides a long-term decay property. As the relative distance $|m-n|$ increases, the expected inner product between tokens generally decreases, helping the model to focus on a more relevant local context while still being capable of long-range dependencies.

The Key Property: The magic of RoPE lies in how it behaves under a dot product. Because the rotation matrix $\mathbf{R}$ is orthogonal, the inner product between a rotated query at position m and a rotated key at position n depends only on their relative distance. Using the property that $\mathbf{R}_m^\top = \mathbf{R}_{-m}$, we can shift the transformation:

$$\langle f(\mathbf{q}, m), f(\mathbf{k}, n)\rangle = (\mathbf{R}_m\mathbf{q})^\top(\mathbf{R}_n\mathbf{k}) = \mathbf{q}^\top\mathbf{R}_m^\top\mathbf{R}n\mathbf{k} = \mathbf{q}^\top\mathbf{R}n - m\mathbf{k} \tag{8}$$

This means the attention score intrinsically depends on the relative distance, not the absolute positions, allowing the model to generalize to much longer sequences than it was trained on. It also has the desirable property that the attention strength naturally decays as the relative distance between tokens increases.

9.3 Computational Efficiency

9.3.1 Sparse Attention

Instead of computing the full, dense attention matrix, sparse attention methods compute attention only for a subset of token pairs. Models like **BigBird [31]** combine several sparse attention patterns:

- **Window Attention:** Each token attends to a local window of neighboring tokens.
- **Global Attention:** A few special tokens (like '[CLS]') attend to all other tokens.
- **Random Attention:** Each token attends to a small, random set of other tokens.

This combination approximates the full attention matrix while reducing the complexity to be linear with sequence length.

9.3.2 FlashAttention [6]

This is not an approximation but a hardware-aware optimization for computing exact attention. The standard implementation of attention involves writing the large $L \times L$ attention matrix to the GPU's high-bandwidth memory (HBM), which is slow. **FlashAttention** restructures the computation to avoid this. It loads blocks of the Q, K, and V matrices into the much faster on-chip SRAM, computes attention for that block, and writes only the final result back to HBM. It uses techniques like tiling and recomputation to make this possible, leading to significant speedups (2-4x) and memory savings.

- **FlashAttention-2 [5]:** FlashAttention reaches only 25-40% of the theoretical maximum FLOPs/s due to suboptimal work partitioning. So, Fashattention-2 further optimizes this by improving parallelism on the GPU.
- **FlashAttention-3 [21]:** Specifically optimized for Hopper (H100) architecture. It exploits hardware asynchrony via warp-specialization. It also leverages FP8 Tensor Cores, providing a throughput boost while minimizing numerical error through incoherent processing.
- **FlashAttention-4 [30]:** Specifically optimized for Blackwell based systems such as the B200 and GB200. It addresses asymmetric scaling where Tensor Core power grew faster than memory bandwidth by redesigning pipelines and developing new scheduling strategies.

9.3.3 Grouped-Query Attention (GQA) [2]

Grouped-Query Attention (GQA) addresses a fundamental hardware bottleneck during autoregressive inference: the Key-Value (KV) cache. The model must store and repeatedly load the keys and values for all previous tokens from GPU memory to compute attention for the next token. As the context window grows, moving this massive KV cache from High-Bandwidth Memory (HBM) to the compute units becomes the primary bottleneck, leaving the GPU's math units underutilized.

To understand GQA, it is best to view it as an interpolation between two architectural extremes:

- **Multi-Head Attention (MHA) [27]:** The standard Transformer approach . For H query heads, there are exactly H key heads and H value heads. This yields the highest model capacity and quality but results in a massive KV cache.

- **Multi-Query Attention (MQA) [22]:** An extreme optimization. All H query heads share a single key head and a single value head. This drastically shrinks the KV cache footprint but it forces a severe dimensional reduction that often degrades model quality.

GQA strikes an optimal balance between the quality of MHA and the speed of MQA. Instead of a 1:1 ratio or an H:1 ratio, the H query heads are divided into G distinct groups. Each group of query heads shares a single key head and a single value head. This significantly reduces GPU memory requirements and increases inference throughput. Because of this favorable trade-off, GQA has become the default attention mechanism in modern large language models.

9.3.4 Linear Attention

These methods aim to reduce the computational complexity of the attention mechanism from quadratic ($O(L^2)$) to linear ($O(L)$) with respect to sequence length L. This is achieved by avoiding the explicit computation of the massive $L \times L$ attention matrix. This increasingly popular family of methods under the umbrella of Linear Attention was first introduced by Katharopoulos et al. (2020) [13].

The Associative Trick: The standard attention computation, Attention $= \text{softmax}(QK^T)V$, is bottlenecked by the matrix multiplication QK^T. Linear attention methods work by replacing the softmax function with a kernelized similarity function, $\text{sim}(Q, K)$, that can be expressed as the dot product of feature maps ϕ:

$$\text{sim}(Q, K) = \phi(Q)\phi(K)^T \tag{9}$$

A common choice for the feature map is the Exponential Linear Unit (ELU) plus one, $\phi(x) = \text{elu}(x) + 1$, to ensure positivity.

This decomposition allows for a crucial change in the order of operations due to the associative property of matrix multiplication. Instead of computing the full attention matrix first:

$$\text{Output} = (\phi(Q)\phi(K)^T)V \tag{10}$$

We can re-associate the terms like this:

$$\text{Output} = \phi(Q)(\phi(K)^T V) \tag{11}$$

Analyzing the complexity of the second approach reveals the linear scaling. The term $(\phi(K)^T V)$ involves multiplying a $(d \times L)$ matrix with an $(L \times d_v)$ matrix, resulting in a small $(d \times d_v)$ matrix. This computation is $O(L \cdot d \cdot d_v)$. The final multiplication of $\phi(Q)$ (an $(L \times d)$ matrix) with this result is also linear in L. The overall complexity is therefore $O(L)$, a dramatic improvement.

Recurrent Formulation and Inference: A significant advantage of this formulation is its equivalence to a Recurrent Neural Network (RNN). The global context vector $S_i = \sum_{j=1}^{i} \phi(\mathbf{k}_j)^T \mathbf{v}_j$ can be computed iteratively:

$$S_i = S_{i-1} + \phi(\mathbf{k}_i)^T \mathbf{v}_i \tag{12}$$

The output at step i is then simply $\mathbf{o}_i = \phi(\mathbf{q}_i)S_i$. This means that during autoregressive inference (generating one token at a time), the state S_{i-1} can be cached and updated in constant time ($O(1)$) to compute the next output. This is a massive speedup compared to standard Transformers, where the entire Key-Value cache for all previous tokens must be re-processed at each step, an $O(L)$ operation.

Chunkwise Formulation: While the recurrent form enables $O(1)$ inference, computing it token-by-token is highly sequential and underutilizes the massive parallel compute capabilities of modern GPUs during training. To strike an optimal balance between the sequential $O(L)$ recurrent form and the hardware-friendly $O(L^2)$ parallel form, we can adopt a chunkwise formulation. This approach splits the input sequence of length L into C distinct chunks, each of length L/C. Within each chunk, attention is computed in parallel, while historical context is passed between chunks recurrently via the hidden state S. Let c denote the chunk index. The hidden state S is updated at the chunk level by aggregating the keys and values entirely within chunk c:

$$S_{[c+1]} = S_{[c]} + \sum_{i=((c-1)\frac{L}{C}+1)}^{c\frac{L}{C}} \phi(\mathbf{k}i)^T \mathbf{v}i = S[c] + \phi(K[c])^T V_{[c]} \tag{13}$$

To compute the output $O_{[c+1]}$ for the next chunk, we combine the historical global context ($S_{[c]}$) with the local attention computed within chunk $c+1$. For the local computation, we apply a standard causal mask M to ensure tokens do not attend to future tokens within the same chunk:

$$O_{[c+1]} = \phi(Q_{[c+1]})S_{[c]} + \left(\phi(Q_{[c+1]})\phi(K_{[c+1]})^T \odot M\right) V_{[c+1]} \tag{14}$$

If C is chosen to be too large, the model is bottlenecked by sequential recurrent updates. Conversely, if C is too small, the quadratic cost of the local attention dominates. Therefore, tuning the number of chunks C is essential to perfectly balance these two terms and achieve the maximum training throughput.

Trade-offs: While extremely efficient, linear attention is an approximation of the powerful softmax attention. The exponential nature of the softmax function allows the model to sharply focus on a few key tokens, effectively ignoring others. Most linear attention kernels are "softer" and do not replicate this property perfectly, which can sometimes lead to a degradation in model performance compared to their exact attention counterparts. Modern architectures today will interleave linear attention layers and softmax attention layers together to reduce the downside.

9.4 Normalization Improvements

- **Pre-LN vs. Post-LN:** The original Transformer used Post-Layer Normalization (applying LN after the residual connection), which can lead to unstable training. Most modern Transformers use **Pre-LN** (applying LN *before* the sub-layer), which stabilizes training and often removes the need for learning rate warm-up.
- **RMS Norm:** Root Mean Square Layer Normalization is a simplified version of LayerNorm. It normalizes the vector by its root-mean-square, but omits the re-scaling step based on learnable parameters. It is more computationally efficient and has been shown to be just as effective in large language models.

10 Transformers in Vision (ViT)

The Transformer's success has been extended to computer vision.

- **Vision Transformer (ViT) [9]:** The core idea is to treat an image as a sequence of fixed-size patches (e.g., 16x16 pixels). These patches are flattened and linearly projected to create "token" embeddings. Paired with positional embeddings, this sequence of tokens is fed into a standard Transformer encoder for a classification task. ViTs have shown that a reliance on convolution is not necessary for state-of-the-art vision results, though they typically require very large pre-training datasets.
- **Swin Transformer [18]:** Swin improves upon ViT by reintroducing some useful inductive biases from CNNs. It computes attention locally within non-overlapping windows and builds a hierarchical representation (feature maps get smaller in deeper layers). To allow for cross-window communication, the windows are shifted in subsequent layers. This design achieves linear computational complexity with respect to image size and performs strongly on a wider range of vision tasks.

11 Beyond Transformers: State-Space Models (SSM)

While Transformers are dominant, research into architectures that scale linearly with sequence length while retaining strong performance is active. State-Space Models are a leading alternative.

- **Concept:** SSMs are inspired by control theory and are fundamentally a type of RNN. A continuous signal $x(t)$ is mapped to a representation $y(t)$ via a latent state $h(t)$. Note that the formulation in papers are usually $u(t)$ for input and $x(t)$ for latent state, but we use a different formulation here so that the connection to recurrent models is more apparent.
- **S4 (Structured State Space for Sequence Modeling) [12]:** The problem with RNNs is their slow, sequential nature. S4 solves this by showing that for discrete, time-invariant systems, the entire recurrent process can be unrolled into a single, large **convolutional kernel**. This

convolution can be computed very quickly in parallel using Fast Fourier Transforms (FFT). A special initialization of the state matrix (HiPPO [11]) allows it to effectively model long-range dependencies.

At its core, S4 maps a 1D input sequence x to an output y through a hidden state h. It begins with continuous parameters $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$, alongside a step size Δ that represents the resolution of the input. Using Δ, the continuous system is discretized into the following recurrent step formulas:

$$h_k = \bar{\mathbf{A}} h_{k-1} + \bar{\mathbf{B}} x_k \tag{15}$$

$$y_k = \bar{\mathbf{C}} h_k \tag{16}$$

where $\bar{\mathbf{A}}$, $\bar{\mathbf{B}}$, and $\bar{\mathbf{C}}$ are discretized matrices. You can think of the $\bar{\mathbf{B}}$ as transforming the input into the latent space, $\bar{\mathbf{A}}$ as a control for the latent space, and $\bar{\mathbf{C}}$ as a transformation for extracting the output. This is similar to RNNs.

- **Mamba [10]:** Mamba improves upon S4. While S4's convolutional filter is input-agnostic, Mamba introduces a **Selective Scan (S6)** mechanism. Its state-space parameters are derived from the input itself, making them time-varying and input-dependent. This allows Mamba to selectively focus on or ignore parts of the input, mimicking the adaptive behavior of attention. Mamba achieves linear scaling in sequence length during training and constant time ($O(1)$) per step during inference, while being competitive with Transformer models.

  To achieve this, Mamba introduces two primary innovations:

  1. The Selection Mechanism: In traditional S4 models, the four parameters $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ are constant across time. Mamba's selection mechanism adds back input-dependent dynamics by making $\Delta, \mathbf{B}$, and $\mathbf{C}$ functions of the input x. Because these parameters now have a sequence length dimension L, the model changes from time-invariant to time-varying. While this flexibility is powerful, it means Mamba loses the equivalence to fast global convolutions (FFTs) used in S4.

  2. Hardware-aware Algorithm: To regain the training speed lost by abandoning convolutions, Mamba leverages a hardware-optimized parallel associative scan. In modern GPUs, most operations (except matrix multiplications) are bottlenecked by memory bandwidth. Instead of computing and materializing the massive discretized scan inputs $(\bar{\mathbf{A}}, \bar{\mathbf{B}})$ in the GPU's slow High-Bandwidth Memory (HBM), Mamba loads the SSM parameters $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ directly into the much faster on-chip SRAM. The discretization and recurrence are performed entirely within SRAM, and only the final output y is written back to HBM.

Mamba Forward Pass Algorithm: To clearly track the flow of data through the scan, let B denote the batch size, L the sequence length (time steps), D the input model dimension, and N the hidden state dimension. The forward pass dynamically generates the parameters from the input and applies the scan iteratively:

Input: $x \in \mathbb{R}^{B \times L \times D}$
Output: $y \in \mathbb{R}^{B \times L \times D}$
1: $\mathbf{A} \in \mathbb{R}^{D \times N} \leftarrow$ Parameter ▷ *Represents structured $N \times N$ matrix*
2: $\mathbf{B} \in \mathbb{R}^{B \times L \times N} \leftarrow s_B(x)$
3: $\mathbf{C} \in \mathbb{R}^{B \times L \times N} \leftarrow s_C(x)$
4: $\Delta \in \mathbb{R}^{B \times L \times D} \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
5: $\bar{\mathbf{A}}, \bar{\mathbf{B}} \in \mathbb{R}^{B \times L \times D \times N} \leftarrow \text{discretize}(\Delta, \mathbf{A}, \mathbf{B})$
6: $y \leftarrow \text{SSM}(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C})(x)$ ▷ *Time-varying: recurrence (scan) only*
7: **return** y

Algorithm Definitions in Code: To bridge the gap between the mathematical pseudo-code and the actual implementation, it is helpful to look at the underlying operations.

1. The Selection Mechanism (s_B, s_C, s_Δ): The functions s_B, s_C, and s_Δ are simply learned transformation matrices applied to the input x. In PyTorch, this is implemented as a standard linear layer (nn.Linear) that projects the input dimension, which is then split into the three respective dynamic parameters:

```
# The 's' functions are executed as a single linear transformation matrix
self.x_proj = nn.Linear(d_inner, dt_rank + d_state * 2, bias=False)

# 1. Project the input x
x_projected = self.x_proj(x)

# 2. Split the projection into Delta (dt), B, and C
dt, B, C = torch.split(x_projected, [dt_rank, d_state, d_state], dim=-1)
```

2. The Discretization Step: Because SSMs originate from continuous-time differential equations, they must be converted into discrete steps for digital hardware. The model uses a Zero-Order Hold (ZOH) approximation to transform the continuous matrices $\mathbf{A}$ and $\mathbf{B}$ into their discrete equivalents $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$. The variable `dt` (representing the step size Δ) acts as a data-dependent gate that controls the resolution of the discretization, allowing the model to dynamically choose whether to focus heavily on the current input or simply maintain its historical state. In PyTorch, this is implemented using `torch.einsum` (the Einstein summation convention), which explicitly defines how tensor dimensions are multiplied.

```
# Discretize A
dA = torch.exp(torch.einsum("bd,dn->bdn", dt, A))

# Discretize B
dB = torch.einsum("bd,bn->bdn", dt, B)
```

During training, to scale linearly with respect to sequence length, Mamba entirely avoids the quadratic bottleneck of standard Transformer attention. Moreover, during autoregressive inference, a standard Transformer, requires attention to all previous tokens to predict the next token . Mamba, conversely, does not need to look back at past tokens because all relevant historical context is compressed and continuously updated within its fixed-size hidden state. As a result, Mamba achieves constant time and constant memory per generation step, regardless of how long the context becomes.

- **Hybrid Models (Jamba [16]):** The latest trend is to combine the strengths of different architectures. **Jamba** is a hybrid model that interleaves blocks of Mamba layers with traditional Transformer attention layers, aiming to get the best of both worlds: the efficiency of Mamba and the proven performance of attention.
- **Connection to linear attention:** Mamba can also be interpreted as a kind of gated linear attention (recall how linear attentions are RNNs) with parameterized decay factor based on the distance of the tokens. Notice how in the selection mechanism, x is projected into dt, B, and C. This is very similar to how self-attention creates QKV drawing some connection between SSMs and transformers. This is explained in detail in [7].

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[3] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode# 5–Workshop on Challenges & Perspectives in Creating Large Language Models*, pages 95–136, 2022.

[4] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of machine learning research*, 24(240):1–113, 2023.

[5] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[6] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

[7] Tri Dao and Albert Gu. Transformers are SSMs: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First conference on language modeling*, 2024.

[11] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. HiPPO: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33:1474–1487, 2020.

[12] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

[13] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[14] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

[15] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7871–7880, 2020.

[16] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[17] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[18] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.

[19] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018.

[20] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[21] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024.

[22] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[23] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MASS: Masked sequence to sequence pre-training for language generation. *arXiv preprint arXiv:1905.02450*, 2019.

[24] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[25] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[26] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[28] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. BLOOM: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

[29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[30] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. *arXiv preprint arXiv:2603.05451*, 2026.

[31] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big Bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.